

Pregunta **4**

Sin responder aún

Puntúa como 1,00

🚩 Marcar
pregunta

En el patron State se produce una doble referencia entre la clase de contexto y las clases estado, ya que la clase de contexto tiene una referencia a la clase "estado" y la clase de estado tiene una referencia a la clase de "contexto"

Seleccione una:

- ☐ Verdadero
- ☐ Falso

Pregunta **3**

Sin responder aún

Puntúa como 1,00

🚩 Marcar pregunta

La serialización es una forma de hacer una clonación

Seleccione una:

- ☐ Verdadero
- ☐ Falso

Siguiente página

Pregunta **2**

Sin responder aún

Puntúa como 1,00

🚩 Marcar
pregunta

Un objeto de tipo Observer solo puede observar a un solo objeto de tipo Observable

Seleccione una:

- ☐ Verdadero
- ☐ Falso

Siguiente página

Pregunta **14**

Sin responder aún

Puntúa como 1,00

🚩 Marcar
pregunta

Cuando un hilo ejecuta un método `synchronized` de un recurso compartido, el resto de los hilos:

Seleccione una:

- ☐ a. No pueden acceder a ningún método `synchronized` de ese recurso, pero si a cualquier método no `synchronized`
- ☐ b. No pueden acceder a ningún método de la clase (del recurso).
- ☐ c. Pueden acceder a cualquier método de la clase (del recurso) siempre que sea `public`
- ☐ d. No pueden acceder a este método pero si a cualquiera de los demás métodos de la clase (del recurso)

Terminar intento...

Pregunta **13**

Sin responder aún

Puntúa como 1,00

 Marcar pregunta

Determine cuáles son las afirmaciones verdaderas en cuanto a la la sentencia wait().

Las respuestas incorrectas tienen penalización

Seleccione una o más de una:

- ☐ a. Su función es bloquear al recurso compartido.
- ☐ b. Su función es avisar al resto de los hilos que ha sucedido un cambio en el estado del recurso compartido
- ☐ c. Su función es "dormir" al hilo.
- ☐ d. Su función específica es evitar que dos o más hilos accedan en forma simultánea a un recurso compartido
- ☐ e. Su función es liberar el recurso compartido

Siguiente página

Pregunta **12**

Sin responder aún

Puntúa como 1,00

 Marcar pregunta

Para poder persistir un objeto utilizando archivos XML es necesario

Seleccione una:

- ☐ a. Ninguna de las anteriores
- ☐ b. Que todos sus métodos sean públicos y sus atributos privados
- ☐ c. Tener un constructor público sin parámetros y getters y setters públicos
- ☐ d. Tener un constructor que nos permita pasar por parámetro los atributos que queremos persistir

Siguiente página

Pregunta **11**

Sin responder aún

Puntúa como 1,00

 Marcar pregunta

Determinar si es de buen diseño la definición de la clase XXX.

```
public class XXX extends Thread
{
    private int dato;
    public synchronized XXX ( int dato )
    {
        this.dato=dato;
    }
    //resto del código
}
```

Seleccione una:

- ☐ Verdadero
- ☐ Falso

Siguiente página

Pregunta **14**

Sin responder aún

Puntúa como 1,00

🚩 Marcar
pregunta

Para poder persistir un objeto utilizando archivos XML es necesario

Seleccione una:

- ☐ a. Ninguna de las anteriores
- ☐ b. Tener un constructor público sin parámetros y getters y setters públicos
- ☐ c. Que todos sus métodos sean públicos y sus atributos privados
- ☐ d. Tener un constructor que nos permita pasar por parámetro los atributos que queremos persistir

Pregunta **10**

Sin responder aún

Puntúa como 1,00

 Marcar
pregunta

Elija la/s respuesta/s correctas. Una elección equivocada tiene penalización.

El patrón State permite:

Seleccione una o más de una:

- ☐ a. Permitir que un objeto cambie el comportamiento al cambiar el estado
- ☐ b. Permite añadir responsabilidades a un objeto en tiempo de ejecución
- ☐ c. Determinar un algoritmo general en la clase base y determinar los comportamientos particulares a través del mecanismo de herencia

Siguiente página

Pregunta **13**

Sin responder aún

Puntúa como 1,00

 Marcar
pregunta

Dada una clase parametrizada, siendo T el identificador del tipo de datos parametrizado:

Seleccione una:

- ☐ a. No se puede usar T como retorno de ningún método.
- ☐ b. No se puede usar T dentro de un método estático o inicializador estático
- ☐ c. Todas las opciones son válidas
- ☐ d. No se puede usar T como parámetro de ningún método.

Pregunta **9**

Sin responder aún

Puntúa como 1,00

 Marcar
pregunta

En el Patrón MVC.

¿De quién es la responsabilidad de validar los datos que el usuario ingresa?

Seleccione una:

- ☐ a. Controlador
- ☐ b. Vista
- ☐ c. Modelo

Siguiente página

Pregunta **8**

Sin responder aún

Puntúa como 1,00

 Marcar pregunta

Se tiene una clase Docente, que se extiende de Persona.

Se necesita que trabaje en forma concurrente (la clase Docente). Para lograr esto y no perder la línea de herencia

Elija todas las opciones correctas.

La opciones incorrectas elegidas como correctas tienen penalización.

Seleccione una o más de una:

- ☐ a. Se debe comenzar la ejecución del objeto de tipo Docente con la instrucción run()
- ☐ b. Se debe implementar la interface Runnable en la clase Docente.
- ☐ c. Se debe crear un nuevo objeto de tipo Thread.
- ☐ d. Se debe implementar la interface Runnable en la clase Persona.

Siguiente página

Elija la/s respuesta/s correctas. Una elección equivocada tiene penalización.

El patrón State permite:

Select one or more:

- ☐ a. Determinar un algoritmo general en la clase base y determinar los comportamientos particulares a través del mecanismo de herencia
- ☐ b. Permite añadir responsabilidades a un objeto en tiempo de ejecución
- ☐ c. Permitir que un objeto cambie el comportamiento al cambiar el estado

Pregunta /

Sin responder aún

Puntúa como 1,00

🚩 Marcar pregunta

Es correcta la implementación de los siguientes métodos?

```
public class Primero
{
    private String nombre;
    private long dni; // identificador que no se repite
    private Date fecha_nac;

    public Primero()
    {
        super();
    }
    @Override
    public boolean equals(Object object)
    {
        if (this == object)
        {
            return true;
        }
        if (!(object instanceof Primero))
        {
            return false;
        }
        final Primero other = (Primero) object;
        if (dni != other.dni)
        {
            return false;
        }
        return true;
    }
    @Override
    public int hashCode()
    {
        final int PRIME = 37;
        int result = 1;
        result = PRIME * result + ((nombre == null) ? 0 : nombre.hashCode());
        result = PRIME * result + (int) (dni ^ (dni >>> 32));
        result = PRIME * result + ((fecha_nac == null) ? 0 : fecha_nac.hashCode
    ());
        return result;
    }
}
```

Pregunta **14**

Sin responder aún

Puntúa como 1,00

🚩 Marcar pregunta

Cuando un hilo libera el recurso compartido el proximo hilo que lo accederá será:

Seleccione una:

- ☐ a. El último hilo que entro a la cola de espera
- ☐ b. El primer hilo que entro a la cola de espera
- ☐ c. Se decide mediante un atributo que determina su prioridad
- ☒ d. Es impredecible, cualquier hilo puede tomar el control del recurso compartido
- ☐ e. El hilo que comenzó su ejecución primero

Terminar intento...

Pregunta **6**

Sin responder aún

Puntúa como 1,00

 Marcar pregunta

Supongamos el caso de un objeto de tipo Observable que está siendo observado por varios objetos de tipo Observer a la vez. En ese contexto, el patrón Observer Observable está pensado para:

Seleccione una o más de una:

- ☐ a. que un objeto observable notifique a un objeto observer a elección.
- ☐ b. Que un objeto observable notifique a todos los objetos observer a la vez.

Siguiente página

Seleccione una:

- ☐ a. la B) admite que dos objetos sean iguales teniendo el mismo dni pero tal vez diferentes nombre y fecha_nac.
- ☐ b. Conviene usar la A) al momento de determinar la igualdad entre dos objetos, conociendo en uno de ellos, solamente uno de los tres atributos.
- ☐ c. La B) representa una situación más real. Es suficiente utilizar el atributo dni para determinar la igualdad de dos objetos dentro de este dominio.
- ☐ d. la A) determina que dos objetos SI son iguales teniendo al menos el mismo dni.

Pregunta **13**

Sin responder aún

Puntúa como 1,00

🚩 Marcar pregunta

Se tiene una clase Docente, que se extiende de Persona.

Se necesita que trabaje en forma concurrente (la clase Docente). Para lograr esto y no perder la línea de herencia

Elija todas las opciones correctas.

La opciones incorrectas elegidas como correctas tienen penalización.

Seleccione una o más de una:

- ☐ a. Se debe implementar la interface Runnable en la clase Docente.
- ☐ b. Se debe comenzar la ejecución del objeto de tipo Docente con la instrucción run()
- ☐ c. Se debe implementar la interface Runnable en la clase Persona.
- ☐ d. Se debe crear un nuevo objeto de tipo Thread.

Siguiente página

Pregunta **8**

Sin responder aún

Puntúa como 1,00

🚩 Marcar pregunta

Se tiene una clase Docente, que se extiende de Persona.

Se necesita que trabaje en forma concurrente (la clase Docente). Para lograr ésto y no perder la línea de herencia

Elija todas las opciones correctas.

La opciones incorrectas elegidas como correctas tienen penalización.

Seleccione una o más de una:

- ☐ a. Se debe implementar la interface Runnable en la clase Docente.
- ☐ b. Se debe implementar la interface Runnable en la clase Persona.
- ☐ c. Se debe comenzar la ejecución del objeto de tipo Docente con la instrucción run()
- ☐ d. Se debe crear un nuevo objeto de tipo Thread.

En el Patrón MVC.

¿De quién es la responsabilidad de validar los datos que el usuario ingresa?

Select one:

- ☐ a. Controlador
- ☐ b. Vista
- ☐ c. Modelo

```
public class Main
{
    public static void main(String args[])
    {
        MiObservable modelo;
        MiObserver ojo;
        modelo = new MiObservable();
        ojo = new MiObserver(modelo);
    }
}
```

```
public class Main2
{
    public static void main(String args[])
    {
        MiObservable modelo;
        MiObserver ojo;
        modelo = new MiObservable();
        ojo = new MiObserver();
        ojo.setUnObservable(modelo);
        modelo.addObserver(ojo);
    }
}
```

Seleccione una:

- ☐ a. La B) representa una situación más real. Es suficiente utilizar el atributo dni para determinar la igualdad de dos objetos dentro de este dominio.
- ☐ b. la A) determina que dos objetos SI son iguales teniendo al menos el mismo dni.
- ☐ c. Conviene usar la A) al momento de determinar la igualdad entre dos objetos, conociendo en uno de ellos, solamente uno de los tres atributos.
- ☐ d. la B) admite que dos objetos sean iguales teniendo el mismo dni pero tal vez diferentes nombre y fecha_nac.

B)

```
public class Primero
{
    private String nombre;
    private long dni; //no se repite
    private Date fecha_nac;
    @Override
    public boolean equals(Object object)
    {
        if (this == object)
        {
            return true;
        }
        if (!(object instanceof Primero))
        {
            return false;
        }
        final Primero other = (Primero) object;
        if (dni != other.dni)
        {
            return false;
        }
        return true;
    }
}
```

Pregunta **5**

Sin responder aún

Puntúa como 1,00

🚩 Marcar pregunta

Considerando las siguientes implementaciones del método equals

A)

```
public class Primero
{
    private String nombre;
    private long dni; // identificador que no se repite
    private Date fecha_nac;
    @Override
    public boolean equals(Object object)
    {
        if (this == object)
        {
            return true;
        }
        if (!(object instanceof Primero))
        {
            return false;
        }
        final Primero other = (Primero) object;
        if (!(nombre == null ? other.nombre == null : nombre.equals(other.nombre)))
        {
            return false;
        }
        if (dni != other.dni)
        {
            return false;
        }
        if (!(fecha_nac == null ? other.fecha_nac == null : fecha_nac.equals(other.fecha_nac)))
        {
            return false;
        }
        return true;
    }
}
```


Pregunta **4**

Sin responder aún

Puntúa como 1,00

 Marcar pregunta

La serialización es una forma de hacer una clonación

Seleccione una:

- ☐ Verdadero
- ☐ Falso

Siguiente página

Dada una clase parametrizada, siendo T el identificador del tipo de datos parametrizado:

Seleccione una:

- ☒ a. No se puede usar T dentro de un método estático o inicializador estático
- ☐ b. No se puede usar T como retorno de ningún método.
- ☐ c. Todas las opciones son válidas
- ☐ d. No se puede usar T como parámetro de ningún método.

Determinar si es de buen diseño la definición de la clase XXX.

```
public class XXX extends Thread
{
    private int dato;
    public synchronized XXX ( int dato )
    {
        this.dato=dato;
    }
    //resto del código
}
```

Seleccione una:

- ☐ Verdadero
- ☐ Falso

Pregunta **6**

Sin responder aún

Puntúa como 1,00

🚩 Marcar pregunta

Determine cuáles son las afirmaciones verdaderas en cuanto a la la sentencia wait().

Las respuestas incorrectas tienen penalización

Seleccione una o más de una:

- ☐ a. Su función es "dormir" al hilo.
- ☐ b. Su función específica es evitar que dos o más hilos accedan en forma simultánea a un recurso compartido
- ☐ c. Su función es liberar el recurso compartido
- ☐ d. Su función es avisar al resto de los hilos que ha sucedido un cambio en el estado del recurso compartido
- ☐ e. Su función es bloquear al recurso compartido.

Siguiente página

Para poder persistir un objeto utilizando archivos XML es necesario

Seleccione una:

- ☐ a. Tener un constructor público sin parámetros y getters y setters públicos
- ☐ b. Ninguna de las anteriores
- ☐ c. Tener un constructor que nos permita pasar por parámetro los atributos que queremos persistir
- ☐ d. Que todos sus métodos sean públicos y sus atributos privados

En el patron State se produce una doble referencia entre la clase de contexto y las clases estado, ya que la clase de contexto tiene una referencia a la clase "estado" y la clase de estado tiene una referencia a la clase de "contexto"

Seleccione una:

- ☐ Verdadero
- ☐ Falso

Pregunta **13**

Sin responder aún

Puntúa como 1,00

 Marcar
pregunta

La serialización es una forma de hacer una clonación

Seleccione una:

- ☐ Verdadero
- ☐ Falso

El siguiente código permite ejecutar concurrentemente dos Tareas: t1 y t2.

```
public class Tarea extends Thread
{
    private char c;
    private int k;
    public Tarea(char c, int k)
    {
        super();
        this.c = c;
        this.k = k;
    }
    . . .
    @Override
    public void run()
    {
        for (int i = 0; i < this.k; i++)
        {
            System.out.println(this.c+" "+i);
        }
    }
}

public class Prueba
{
    public static void main(String[] args)
    {
        Tarea t1=new Tarea("A",4);
        Tarea t2=new Tarea("B",3);
        t1.run();
        t2.run();
    }
}
```


Cuando un hilo ejecuta un método synchronized de un recurso compartido, el resto de los hilos:

Select one:

- ☐ a. No pueden acceder a este método pero si a cualquiera de los demás métodos de la clase (del recurso)
- ☒ b. No pueden acceder a ningún método synchronized de ese recurso, pero si a cualquier método no synchronized
- ☐ c. No pueden acceder a ningún método de la clase (del recurso).
- ☐ d. Pueden acceder a cualquier método de la clase (del recurso) siempre que sea public

Se tiene una clase Docente, que se extiende de Persona.

Se necesita que trabaje en forma concurrente (la clase Docente). Para lograr ésto y no perder la línea de herencia

Elija todas las opciones correctas.

La opciones incorrectas elegidas como correctas tienen penalización.

Select one or more:

- ☐ a. Se debe implementar la interface Runnable en la clase Persona.
- ☒ b. Se debe implementar la interface Runnable en la clase Docente.
- ☒ c. Se debe crear un nuevo objeto de tipo Thread.
- ☐ d. Se debe comenzar la ejecución del objeto de tipo Docente con la instrucción run()

```
return true;
}
@Override
public int hashCode()
{
    final int PRIME = 37;
    int result = 1;
    result = PRIME * result + ((nombre == null) ? 0 : nombre.hashCode());
    result = PRIME * result + (int) (dni ^ (dni >>> 32));
    result = PRIME * result + ((fecha_nac == null) ? 0 : fecha_nac.hashCode());
    return result;
}
}
```

Seleccione una:

- ☐ Verdadero
- ☐ Falso

```
public class Primero
{
    private String nombre;
    private long dni; // identificador que no se repite
    private Date fecha_nac;

    public Primero()
    {
        super();
    }
    @Override
    public boolean equals(Object object)
    {
        if (this == object)
        {
            return true;
        }
        if (!(object instanceof Primero))
        {
            return false;
        }
        final Primero other = (Primero) object;
        if (dni != other.dni)
        {
            return false;
        }
        return true;
    }
    @Override
    public int hashCode()
    {
        final int PRIME = 37;
        int result = 1;
```

Seleccione una o más de una:

- ☐ a. Main1 y Main2 tienen códigos correctos y el diseño de Main2 es mejor que el de Main1 debido a un tema de asignación de responsabilidades y prevención de errores
- ☐ b. el código de la clase Main2 no funciona
- ☐ c. Main1 y Main2 tienen códigos correctos y el diseño de Main1 es mejor que el de Main2 debido a un tema de asignación de responsabilidades y prevención de errores

Se tiene una clase Docente, que se extiende de Persona.

Se necesita que trabaje en forma concurrente (la clase Docente). Para lograr ésto y no perder la línea de herencia

Elija todas las opciones correctas.

La opciones incorrectas elegidas como correctas tienen penalización.

Select one or more:

- ☐ a. Se debe implementar la interface Runnable en la clase Persona.
- ☐ b. Se debe implementar la interface Runnable en la clase Docente.
- ☐ c. Se debe crear un nuevo objeto de tipo Thread.
- ☐ d. Se debe comenzar la ejecución del objeto de tipo Docente con la instrucción run()

De acuerdo al siguiente enunciado elija las opciones correctas. Las opciones incorrectas elegidas como correcta tienen penalización.

Tenga en cuenta que existen dos clases principales que plantean dos formas diferentes de utilizar los métodos de ambos objetos

```
public class MiObserver implements Observer
{
    private MiObservable unObservable;
    //otros atributos

    public MiObserver()
    {
        //inicialización de otros atributos
    }

    public MiObserver(MiObservable unObservable)
    {
        this.setUnObservable(unObservable);
        this.getUnObservable().addObserver(this);
        //inicialización de otros atributos
    }

    @Override
    public void update(Observable o, Object arg)
    {
        // TODO Implement this method
    }

    public MiObservable getUnObservable()
    {
        return unObservable;
    }
}
```

```
public class MiObservable extends Observable
{
    //atributos específicos

    public MiObservable()
    {
        super();
    }
}
```

```
public class Main
{
    public static void main(String args[])
    {
        MiObservable modelo;
        MiObserver ojo;
        modelo = new MiObservable();
        ojo = new MiObserver(modelo);
    }
}
```

```
public class Main2
{
    public static void main(String args[])
    {
        MiObservable modelo;
        MiObserver ojo;
        modelo = new MiObservable();
        ojo = new MiObserver();
        ojo.setUnObservable(modelo);
        modelo.addObserver(ojo);
    }
}
```

Seleccione una o más de una:

- ☐ a. Main1 y Main2 tienen códigos correctos y el diseño de Main2 es mejor que el de Main1 debido a un tema de asignación de responsabilidades y prevención de errores
- ☐ b. el código de la clase Main2 no funciona
- ☐ c. Main1 y Main2 tienen códigos correctos y el diseño de Main1 es mejor que el de Main2 debido a un tema de asignación de responsabilidades y prevención de errores

Seleccione una o más de una:

- ☐ a. Main1 y Main2 tienen códigos correctos y el diseño de Main1 es mejor que el de Main2 debido a un tema de asignación de responsabilidades y prevención de errores
- ☐ b. Main1 y Main2 tienen códigos correctos y el diseño de Main2 es mejor que el de Main1 debido a un tema de asignación de responsabilidades y prevención de errores
- ☐ c. el código de la clase Main2 no funciona

Pregunta 1

Sin responder aún

Puntúa como 1,00

🚩 Marcar pregunta

Supongamos el caso de un objeto de tipo Observable que está siendo observado por varios objetos de tipo Observer a la vez. En ese contexto, el patrón Observer Observable está pensado para:

Seleccione una o más de una:

- ☐ a. que un objeto observable notifique a un objeto observer a elección.
- ☐ b. Que un objeto observable notifique a todos los objetos observer a la vez.

Siguiente página

Dada la clase genérica Caja<T>, siendo T el identificador del tipo de datos parametrizado, determine el valor de verdad de la variable rta:

```
Caja<Juguete> cajaJuguete = new Caja<Juguete>();  
Caja<Herramienta> cajaH = new Caja<Herramienta>();  
boolean rta = cajaJuguete.getClass() != cajaHerramienta().getClass();
```

Select one:

- ☐ True
- ☐ False